



I am trying to train a ML model to classify Pokemon into smogon tiers. I am using optuna for hyperparameter tuning, there is a number of categorical features (impactful abilities) that will be one-hot encoded but I need to check if said ability exists in a training fold when using cross validation to include it as a feature (in order to avoid data leakage) how can I integrate this into my optuna study?



Gemini 3 Flash Thinking

This is a classic machine learning trap, and you are spot-on for catching it early. Classifying Pokémon into Smogon tiers (like OU, UU, RU) based on stats, typing, and abilities is a great project, but abilities can absolutely cause data leakage if handled incorrectly.

To ensure that your one-hot encoding (OHE) only reflects the abilities present in each specific training fold, you need to bake the preprocessing directly into the cross-validation loop inside your Optuna objective function.

The most robust way to achieve this is by using **scikit-learn's Pipeline** paired with `OneHotEncoder(handle_unknown='ignore')`.

Here is how it works and how to integrate it into your Optuna study.

Why this solves your problem

When you use `OneHotEncoder(handle_unknown='ignore')` inside a pipeline:

1. **During training:** The encoder is fit *only* on the training fold. It maps only the abilities present in that fold to new OHE columns.
2. **During validation:** The encoder transforms the validation fold. If a Pokémon in the validation fold has a rare ability that never appeared in the training fold, `handle_unknown='ignore'` ensures the encoder doesn't crash—it simply outputs all zeros for that Pokémon's OHE columns.

This completely eliminates data leakage because the model never "knows" about categories outside its current training split.

Step-by-Step Implementation

Here is a clean template demonstrating how to set this up using Optuna and an XGBoost or LightGBM classifier.

python

```
import numpy as np
import pandas as pd
import optuna
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from lightgbm import LGBMClassifier # Or XGBClassifier, RandomForestClassifier, etc.

# --- 1. Dummy Data Setup (Replace this with your actual Pokémon dataset) ---
# Let's assume X has numerical stats and a categorical 'ability' column
# y contains the Smogon tiers (encoded as integers 0, 1, 2...)
X = pd.DataFrame({
    'hp': [45, 60, 80, 78, 35, 90, 100],
    'attack': [49, 62, 82, 84, 55, 130, 100],
    'ability': ['Overgrow', 'Overgrow', 'Overgrow', 'Blaze', 'Static', 'Intimidate', 'Huge
Power']
})
y = np.array([0, 0, 1, 0, 0, 2, 2]) # Example target tiers

# Identify your feature types
categorical_features = ['ability']
numerical_features = ['hp', 'attack']

# --- 2. Define the Optuna Objective Function ---
def objective(trial):
    # Suggest hyperparameters for your model
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42,
        'verbosity': -1
    }

    # Define the preprocessing steps
    # 'handle_unknown="ignore"' is the secret sauce that prevents leakage
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), numerical_features),
            ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
categorical_features)
        ]
    )
```

```

)

# Combine preprocessing and the model into a single Pipeline
model = LGBMClassifier(**params)
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', model)
])

# Set up Stratified K-Fold CV (ideal for tier classification)
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# cross_val_score automatically fits the pipeline steps ONLY on the training folds
# and transforms the validation fold dynamically.
scores = cross_val_score(
    pipeline, X, y,
    cv=cv,
    scoring='f1_macro', # Use macro F1 if tiers are imbalanced
    n_jobs=-1
)

# Return the average score across all folds
return np.mean(scores)

# --- 3. Run the Optuna Study ---
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)

print("Best Trial:")
print(study.best_trial.value)
print(study.best_trial.params)

```

Key Points to Keep in Mind

- **The Power of `cross_val_score`** : By passing the entire `Pipeline` object to `cross_val_score`, scikit-learn handles the internal splitting logic. It splits the data, runs `.fit_transform()` on the training split's `OneHotEncoder`, and then runs `.transform()` on the validation split.
- **Handling Multiple Abilities**: If your dataset treats abilities as a multi-label problem (e.g., a Pokémon can have `Ability_1`, `Ability_2`, or a `Hidden_Ability`), you can pass all of those column names into the `categorical_features` list. The `ColumnTransformer` will apply the encoder to all of them seamlessly.
- **Feature Pruning (Optional)**: If you only want "impactful" abilities and want Optuna to decide how rare an ability needs to be to get dropped, you can add `min_frequency` to your `OneHotEncoder` and tune it with Optuna:

python

```
min_freq = trial.suggest_float('min_freq', 0.01, 0.1)
# Drop abilities that appear in less than X% of the training fold data
OneHotEncoder(handle_unknown='ignore', min_frequency=min_freq)
```

2026-05-15 23:05:45 You Asked 

the problem is each pokemon can have up to 3 abilities so the impactful abilities features are not mutually exclusive

Gemini 3 Flash Thinking

Ah, the multi-slot ability dilemma! You are totally right to call this out. Because a Pokémon can have up to three potential abilities (Slot 1, Slot 2, and a Hidden Ability), standard One-Hot Encoding falls apart. If "Intimidate" is in Slot 1 for Arcanine but Slot 2 for Gyarados, a standard encoder treats them as two completely separate features (ability1_Intimidate and ability2_Intimidate).

What you actually need is a **Multi-Hot Encoder** (an indicator matrix) that looks across all three ability columns simultaneously, flattens them into a single "pool" of learned abilities during the training fold, and safely ignores unseen abilities during the validation fold.

To integrate this cleanly into your Optuna study without data leakage, you can build a lightweight, custom Scikit-Learn transformer.

The Solution: Custom Multi-Label Ability Encoder

This custom transformer fits perfectly into a Scikit-Learn Pipeline . It learns the unique vocabulary of abilities *only* from the training fold, and maps both training and validation folds to those exact features.

python

```
import numpy as np
import pandas as pd
import optuna
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from lightgbm import LGBMClassifier
```

```

# --- 1. Custom Multi-Hot Encoder for Pokémon Abilities ---
class MultiLabelAbilityEncoder(BaseEstimator, TransformerMixin):
    def __init__(self):
        self.all_abilities_ = []

    def fit(self, X, y=None):
        # Convert input to DataFrame to safely flatten it
        X_df = pd.DataFrame(X)
        # Flatten all slots, extract unique values, and filter out missing/placeholder data
        unique_vals = pd.unique(X_df.values.ravel())
        self.all_abilities_ = [
            a for a in unique_vals
            if pd.notna(a) and a not in ['', 'None', 'nan', None]
        ]
        self.all_abilities_.sort()
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X)
        # Initialize an empty binary matrix (rows = pokemon, cols = learned abilities)
        encoded = np.zeros((len(X_df), len(self.all_abilities_)))

        # Check if a learned ability exists ANYWHERE across the 3 columns for each row
        for idx, ability in enumerate(self.all_abilities_):
            encoded[:, idx] = X_df.isin([ability]).any(axis=1).astype(int)

        return encoded

    def get_feature_names_out(self, input_features=None):
        return np.array(self.all_abilities_)

# --- 2. Mock Dataset Setup ---
# Each Pokémon now has 3 potential ability slots (can include NaNs or None)
X = pd.DataFrame({
    'hp': [45, 60, 80, 78, 35, 90, 100],
    'attack': [49, 62, 82, 84, 55, 130, 100],
    'ability_1': ['Overgrow', 'Overgrow', 'Overgrow', 'Blaze', 'Static', 'Intimidate', 'Huge
Power'],
    'ability_2': [None, 'Chlorophyll', 'Chlorophyll', 'Solar Power', 'Lightning Rod', 'Moxie',
None],
    'ability_hidden': ['Chlorophyll', None, None, None, 'Plus', 'Rivalry', 'Sap Sipper']
})
y = np.array([0, 0, 1, 0, 0, 2, 2]) # Target Smogon Tiers

# Group your features
ability_cols = ['ability_1', 'ability_2', 'ability_hidden']

```

```

numerical_cols = ['hp', 'attack']

# --- 3. Optuna Objective Function ---
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42,
        'verbosity': -1
    }

    # ColumnTransformer directs the 3 ability columns into our custom encoder
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), numerical_cols),
            ('abilities', MultiLabelAbilityEncoder(), ability_cols)
        ]
    )

    # Build the pipeline
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', LGBMClassifier(**params))
    ])

    # Cross-validation handles the folding
    cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
    scores = cross_val_score(pipeline, X, y, cv=cv, scoring='f1_macro', n_jobs=-1)

    return np.mean(scores)

# --- 4. Run Study ---
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=10)

print("Best Hyperparameters:", study.best_trial.params)

```

Why this prevents Data Leakage

1. **Isolation in fit** : When `cross_val_score` cuts your data into folds, the pipeline calls `.fit()` on the custom encoder using **only** the training fold. If a wild validation-fold Pokémon happens to have a completely unique ability (like *As One* or *Delta Stream*) that isn't present anywhere in the training fold, the encoder completely ignores it. It won't create a column for it.

2. **Alignment in transform** : When the validation fold passes through `.transform()` , the `.isin([ability]).any(axis=1)` loop checks the validation rows *only* against the column categories established by the training fold. The unseen validation ability safely yields `0` across the board, completely insulating your model from future data leakage.

2026-05-15 23:21:55 You Asked 

right now the following code has some issues such as `X_gpu` and `y_gpu` being used in the fit function while `X_train_final` is unused.

Can you adapt this optuna study to correctly use gpu: import cupy as cp

```
import xgboost as xgb
```

```
from sklearn.model_selection import StratifiedKFold
```

```
from sklearn.metrics import roc_auc_score
```

```
X_gpu = cp.asarray(X.values)
```

```
y_gpu = cp.asarray(y.values)
```

```
def objective_gpu_cupy(trial):
```

```
    param = {
```

```
        "max_depth": trial.suggest_int("max_depth", 3, 10),
```

```
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1),
```

```
        "n_estimators": trial.suggest_int("n_estimators", 100, 1000),
```

```
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
```

```
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
```

```
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
```

```
        "gamma": trial.suggest_float("gamma", 0, 5),
```

```
        "device": "cuda",
```

```
    }
```

```
    skf = StratifiedKFold(n_splits=5, shuffle=True)
```

```
    scores = []
```

```
    for tr, va in skf.split(
```

```
        cp.asnumpy(X_gpu), cp.asnumpy(y_gpu)
```

```
    ):
```

```
        X_train, X_test = X.iloc[train_index].copy(),
```

```
        X.iloc[test_index].copy()
```

```

y_train, y_test = y.iloc[train_index], y.iloc[test_index]

for ability in impactful_abilities:
    X_train[f"{ability}"] = X_train[['Ability_1', 'Ability_2',
    'Ability_3']].eq(ab).any(axis=1).astype(int)
    X_test[f"{ability}"] = X_test[['Ability_1', 'Ability_2',
    'Ability_3']].eq(ab).any(axis=1).astype(int)

    X_train_final = X_train.drop(columns=['Pokemon', 'Ability_1',
    'Ability_2', 'Ability_3'])
    X_test_final = X_test.drop(columns=['Pokemon', 'Ability_1',
    'Ability_2', 'Ability_3'])

    model = xgb.XGBClassifier(**param)
    model.fit(X_gpu[tr], y_gpu[tr])

    p = model.predict_proba(X_gpu[va])[:, 1]

    scores.append(f1_macro_score(cp.asnumpy(y_gpu[va]),
    cp.asnumpy(p)))
    return float(cp.asarray(scores).mean())

```

Gemini 3 Flash Thinking

Here is the corrected and optimized Optuna study.

The main issue with trying to create `X_gpu = cp.asarray(X.values)` globally is that **CuPy arrays cannot hold strings** (like Pokémon names or text abilities). We need to perform the string-based multi-hot mapping on the CPU using Pandas first, and then push the finalized, purely numeric datasets (`X_train_final` and `X_test_final`) onto the GPU using CuPy right before training.

This code fixes the unused data bugs, resolves the undefined indexing variables, corrects the `ab` typo, fixes the data leakage by calculating the ability pool dynamically per fold, and properly routes everything to your GPU.

python

```

import cupy as cp
import xgboost as xgb
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold

```



```

from sklearn.metrics import f1_score

# Note: Keep X and y as standard Pandas DataFrames/Series globally
# because they contain string data ('Pokemon', 'Ability_1', etc.)

def objective_gpu_cupy(trial):
    param = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1),
        "n_estimators": trial.suggest_int("n_estimators", 100, 1000),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
        "gamma": trial.suggest_float("gamma", 0, 5),
        "device": "cuda", # Crucial: Forces XGBoost to use the GPU
        "eval_metric": "mlogloss", # Standard for multiclass Smogon tiers
        "random_state": 42,
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    # Split using the CPU dataframes since they contain strings
    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

        # --- 1. Dynamic Leakage Prevention ---
        # Find unique abilities present ONLY in this specific training fold
        ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
        unique_abilities = pd.unique(X_train[ability_cols].values.ravel())

        # Filter out NaN/None/empty strings
        fold_abilities = [a for a in unique_abilities if pd.notna(a) and a not in ['', 'None',
None]]

        # --- 2. Vectorized Multi-Hot Encoding ---
        for ability in fold_abilities:
            X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
            X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

        # Drop non-numeric features to prepare for CuPy
        X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
        X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

        # --- 3. Push Processed Numeric Data to GPU ---
        X_train_gpu = cp.asarray(X_train_final.values, dtype=cp.float32)
        X_test_gpu = cp.asarray(X_test_final.values, dtype=cp.float32)

```

```

y_train_gpu = cp.asarray(y_train.values)
y_test_gpu = cp.asarray(y_test.values)

# --- 4. GPU Training & Prediction ---
model = xgb.XGBClassifier(**param)
model.fit(X_train_gpu, y_train_gpu)

# Use predict() instead of predict_proba() since Macro F1 requires hard class labels
preds_gpu = model.predict(X_test_gpu)

# --- 5. Calculate Score ---
# Pull data back to CPU to evaluate with standard Scikit-Learn metrics
y_test_cpu = cp.asnumpy(y_test_gpu)
preds_cpu = cp.asnumpy(preds_gpu)

scores.append(f1_score(y_test_cpu, preds_cpu, average='macro'))

return float(np.mean(scores))

```

Key Improvements Made:

- **Fixed Data Leakage:** The code now scans `X_train` *dynamically inside the loop* to find available abilities for that fold. This ensures your validation set features strictly mimic real-world deployment.
- **Fixed Indexing Variables:** Replaced the undefined `train_index` and `test_index` variables with your loop variables `tr` and `va` .
- **Corrected the Typo:** Fixed `.eq(ab)` to `.eq(ability)` .
- **F1 Macro Alignment:** Changed `predict_proba()[:, 1]` to `predict()` . Because Smogon tiers are a multiclass target (OU, UU, RU, etc.), slicing the second column of a probability matrix would cause an evaluation error. `predict()` returns the actual predicted tier integers, which `f1_score(..., average='macro')` expects.

2026-05-15 23:29:06

You Asked 

why don't we define the number of classes and objective for xgboost in param?

That is a very sharp observation, and thinking about those parameters shows great defensive programming habits!

The short answer is: **We don't have to define them because scikit-learn's `XGBClassifier` wrapper is smart enough to infer them automatically.** Here is exactly what is happening under the hood and why you might still want to add them explicitly.

How XGBoost Handles It Automatically

When you use the native API (`xgb.train`), you are required to define the objective and class count. However, when you use the Scikit-Learn API wrapper (`xgb.XGBClassifier`), the `.fit(X, y)` method inspects your target array `y` right before training begins:

1. **num_class Inference:** It counts the number of unique integers in your `y_train` array and assigns that value to `num_class` behind the scenes.
2. **objective Inference:** If it detects only two unique classes (0 and 1), it defaults to `binary:logistic`. If it detects three or more unique classes (e.g., 0, 1, 2 for Smogon tiers), it automatically switches its internal objective to `multi:softprob`.

Because of this auto-detection, leaving them out keeps your Optuna dictionary clean and prevents hardcoding errors if you ever decide to add or remove tiers (like adding "Ubers" or removing "Little Cup") from your dataset.

Why You Might Want to Explicitly Define Them Anyway

Even though it's automatic, there are two very good reasons to explicitly define them in your `param` dictionary:

1. **Handling Missing Classes in Folds:** If you have a very rare tier (like *Borderline* tiers or *ZU*) and a specific cross-validation fold accidentally leaves that tier completely out of the training split, XGBoost's automatic inference will guess the wrong `num_class`. This will throw a shape mismatch error when it tries to predict on the validation fold.
2. **Explicit is Better Than Implicit:** It makes your code self-documenting and guarantees that XGBoost won't make a wrong assumption.

How to add them to your Optuna Study

If you want to be 100% safe against the fold-omission issue, you can find the total number of unique classes from your global `y` array and pass it explicitly into the parameters like this:

```
python

# Determine the number of unique tiers globally
num_tiers = int(y.nunique()) # or len(np.unique(y))
```

```
def objective_gpu_cupy(trial):
    param = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1),
        "n_estimators": trial.suggest_int("n_estimators", 100, 1000),

        # Explicitly setting the multiclass parameters:
        "objective": "multi:softprob",
        "num_class": num_tiers,

        "device": "cuda",
        "eval_metric": "mlogloss",
        "random_state": 42,
    }

    # ... rest of your cross-validation loop stays exactly the same
```

Using `multi:softprob` is ideal here because even though you ultimately want hard class predictions for your F1 score, `multi:softprob` outputs class probabilities internally, which allows the evaluation metric (`mlogloss`) to calculate smooth gradients during the GPU training process.

2026-05-15 23:40:47 You Asked 

```
ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
unique_abilities = pd.unique(X_train[ability_cols].values.ravel())

fold_abilities = [a for a in unique_abilities if pd.notna(a) and a
not in ['', 'None', None]] this is useless I need to know which
impactful abilities (a list I have defined) is present in fold_abilities
```

Gemini 3 Flash Thinking

Ah, got it. My bad—I completely missed that you already had a specific `impactful_abilities` shortlist defined and only wanted to build features from *that* list, rather than extracting every single random ability in the game.

To fix this, you can use a **set intersection**. You take the set of all unique abilities physically present in the training fold, and intersect it with your predefined `impactful_abilities` list.

This ensures you only build features for your chosen impactful abilities, but safely drops any ability from that shortlist if it happens to be missing from the current training fold (preventing data leakage and keeping your column shapes perfectly aligned between train and test).

Replace that section of the loop with this:

python

```
# --- 1. Dynamic Leakage Prevention (Shortlist Intersection) ---
ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']

# Get a set of all abilities that actually exist in this training fold
abilities_in_fold = set(X_train[ability_cols].values.ravel())

# Filter your predefined list to ONLY include what the training fold has seen
abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]

# --- 2. Vectorized Multi-Hot Encoding ---
# Only create columns for the impactful abilities present in this training fold
for ability in abilities_to_encode:
    X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
    X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)
```

Why this is exactly what you need:

- **No Shape Mismatches:** Because `abilities_to_encode` is determined entirely by `X_train`, both `X_train` and `X_test` will get the exact same columns generated in the exact same order. XGBoost won't complain about mismatched feature counts.
- **No Spurious Features:** If an ability on your impactful list (e.g., *As One*) isn't present in the training fold, it won't create a useless column of all 0s for the model to waste time on.
- **Leakage Proof:** If that missing impactful ability *does* appear in the validation fold, it is completely ignored because it wasn't in `abilities_to_encode`. It won't leak future information into the current fold's training process.

2026-05-15 23:44:57 You Asked 

```
File "/usr/local/lib/python3.12/dist-
packages/optuna/study/_optimize.py", line 206, in _run_trial
    value_or_values = func(trial)
```

^^^^^^^^^^^^

```
File "/tmp/ipykernel_4332/3104107854.py", line 47, in
objective_gpu
```

```
    model.fit(X_train_gpu, y_train_gpu)
```

```
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 751, in inner_f
```

```
    return func(**kwargs)
```

^^^^^^^^^^^^^^^^

File "/usr/local/lib/python3.12/dist-packages/xgboost/sklearn.py", line 1750, in fit
classes = cp.unique(y)

^^^^^^^^^^^^^^^^

File "/usr/local/lib/python3.12/dist-packages/cupy/_manipulation/add_remove.py", line 237, in unique
ret = _unique_1d(ar, return_index=return_index,

^^

File "/usr/local/lib/python3.12/dist-packages/cupy/_manipulation/add_remove.py", line 333, in _unique_1d

ar = cupy.asarray(ar).flatten()

^^

File "cupy/_core/core.pyx", line 988, in cupy._core.core._ndarray_base.flatten

File "cupy/_core/core.pyx", line 1007, in cupy._core.core._ndarray_base.flatten

File "cupy/_core/_routines_manipulation.pyx", line 130, in cupy._core._routines_manipulation._ndarray_flatten

File "cupy/_core/_routines_manipulation.pyx", line 139, in cupy._core._routines_manipulation._ndarray_flatten_order_c

File "cupy/_core/core.pyx", line 822, in cupy._core.core._ndarray_base.copy

File "cupy/_core/core.pyx", line 792, in cupy._core.core._ndarray_base.astype

File "cupy/_core/core.pyx", line 761, in cupy._core.core._ndarray_base._astype

File "cupy/_core/_kernel.pyx", line 1380, in cupy._core._kernel.ufunc.__call__

File "cupy/_core/_kernel.pyx", line 1407, in cupy._core._kernel.ufunc._get_ufunc_kernel

File "cupy/_core/_kernel.pyx", line 1077, in cupy._core._kernel._get_ufunc_kernel

File "cupy/_core/_kernel.pyx", line 103, in cupy._core._kernel._get_simple_elementwise_kernel

File "cupy/_core/_kernel.pyx", line 91, in cupy._core._kernel._get_simple_elementwise_kernel_from_code

File "cupy/_core/core.pyx", line 2550, in cupy._core.core.compile_with_cache

```
File "cupy/_core/core.pyx", line 2514, in
cupy._core.core.assemble_cupy_compiler_options
RuntimeError: Failed to auto-detect CUDA root directory. Please
specify `CUDA_PATH` environment variable if you are using CUDA
versions not yet supported by CuPy.
[W 2026-05-15 20:44:27,167] Trial 0 failed with value None.
[I 2026-05-15 20:44:34,445] Trial 1 finished with value:
0.2620634632013771 and parameters: {'max_depth': 4,
'learning_rate': 0.08758089589708382, 'n_estimators': 321,
'subsample': 0.5525910958003228, 'colsample_bytree':
0.9769904559230951, 'min_child_weight': 1, 'gamma':
2.998933326272915}. Best is trial 1 with value:
0.2620634632013771.
```

Gemini 3 Flash Thinking

This error is a classic environment headache. CuPy is throwing a tantrum because it's trying to compile a CUDA kernel on the fly (specifically when XGBoost calls `cp.unique(y)` under the hood), but it can't locate your system's underlying CUDA Toolkit directory (`nvcc` , library paths, etc.).

The best fix is actually a beautifully simple shortcut: **Drop CuPy entirely.**

Because you have `"device": "cuda"` set in your XGBoost parameters, **XGBoost will natively handle all GPU acceleration itself.** You can feed it standard CPU NumPy arrays or Pandas DataFrames, and XGBoost will automatically transfer the data to the GPU internally.

By removing CuPy from the data-loading step, you completely bypass CuPy's strict path-detection requirements without losing any training speed.

The CuPy-Free, GPU-Enabled Objective Function

Here is the updated code. Notice that we import standard `numpy` instead of `cupy` , and let XGBoost do the heavy lifting on the GPU.

```
python

import xgboost as xgb
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score

def objective_gpu_pure_xgb(trial):
    param = {
```

```

"max_depth": trial.suggest_int("max_depth", 3, 10),
"learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1),
"n_estimators": trial.suggest_int("n_estimators", 100, 1000),
"subsample": trial.suggest_float("subsample", 0.5, 1.0),
"colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
"min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
"gamma": trial.suggest_float("gamma", 0, 5),

# XGBoost handles the GPU internally-no CuPy arrays required!
"device": "cuda",
"eval_metric": "mlogloss",
"random_state": 42,
}

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = []

for tr, va in skf.split(X, y):
    X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
    y_train, y_test = y.iloc[tr], y.iloc[va]

    # --- 1. Dynamic Leakage Prevention ---
    ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
    abilities_in_fold = set(X_train[ability_cols].values.ravel())
    abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]

    # --- 2. Multi-Hot Encoding ---
    for ability in abilities_to_encode:
        X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
        X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

    # Drop non-numeric features
    X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
    X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

    # --- 3. Standard NumPy Conversion ---
    # We pass standard arrays. XGBoost copies these to the GPU instantly.
    X_train_np = X_train_final.to_numpy(dtype=np.float32)
    X_test_np = X_test_final.to_numpy(dtype=np.float32)
    y_train_np = y_train.to_numpy()
    y_test_np = y_test.to_numpy()

    # --- 4. GPU Training & Prediction ---
    model = xgb.XGBClassifier(**param)
    model.fit(X_train_np, y_train_np)

    preds_np = model.predict(X_test_np)

```



```
# --- 5. Calculate Score ---
scores.append(f1_score(y_test_np, preds_np, average='macro'))

return float(np.mean(scores))
```

Why this fixes the issue:

- **No JIT Compilation Failures:** The crash occurred because CuPy tried to compile an internal C++/CUDA routine using an environment it couldn't see. Since XGBoost comes pre-compiled with its own standalone CUDA binaries, it doesn't suffer from this auto-detection weakness.
- **Cleaner Code:** You no longer have to constantly juggle data back and forth using `cp.asarray()` and `cp.asnumpy()`.
- **Performance:** The data transfer overhead from CPU RAM to GPU VRAM happens seamlessly inside `model.fit()`, which is incredibly fast for tabular data like Pokémon stats and abilities.

2026-05-16 11:41:46 You Asked 

I would like to try a SVM approach as well, how can I use SVM for a 7 way classification problem and which hyperparameter should I be mindful of when running an optuna study for it?

Gemini 3 Flash Thinking

Switching to a Support Vector Machine (SVM) for a 7-way classification problem is a solid strategy. While SVMs are inherently binary classifiers, Scikit-learn's `SVC` handles multiclass classification completely automatically under the hood using a **One-vs-One (OvO)** strategy.

For your 7 Smogon tiers, it will train 21 internal binary classifiers (every possible pair of tiers) and use a majority-vote system to pick the final tier. You don't have to change anything structural about your target variable; `SVC` handles the rest.

However, unlike XGBoost, **SVMs are incredibly sensitive to feature scaling and require different hyperparameter tuning.** —

Crucial Rule for SVMs: Feature Scaling

XGBoost doesn't care if a Pokémon's Attack stat is 130 and its multi-hot ability feature is 1. **SVM cares immensely.** Because SVM calculates geometric distances between data points, unscaled features will allow base stats to completely drown out your ability features.

You must scale your numerical features (HP, Attack, Speed, etc.) inside the cross-validation loop to avoid data leakage.

Key SVM Hyperparameters to Tune in Optuna

- **C (Regularization parameter)**: Controls the trade-off between a smooth decision boundary and classifying training points correctly.
 - *Low C*: Smooth boundary, highly generalized, ignores misclassifications (higher bias, lower variance).
 - *High C*: Tight boundary, aims to classify all training points perfectly (lower bias, higher variance/overfitting).
 - *Search space*: Best searched on a logarithmic scale (e.g., 10^{-3} to 10^3).
 - **kernel** : The mathematical function used to map your data into a higher-dimensional space. For complex tabular data like Pokémon data, **rbf** (Radial Basis Function) is usually the golden standard, but it is worth letting Optuna try **linear** or **poly** .
 - **gamma (Kernel coefficient)**: Only applies to **rbf** , **poly** , and **sigmoid** . It defines how far the influence of a single training example reaches.
 - *Low gamma*: Large radius of influence; points far apart are considered close. Smooth boundaries.
 - *High gamma*: Small radius of influence; points must be very close to affect each other. Creates complex, "island-like" decision boundaries (prone to overfitting).
 - *Search space*: Best searched on a logarithmic scale (e.g., 10^{-4} to 10^1).
-

Optuna Implementation for SVM

Because a Pokémon dataset is typically small (around 1,000 unique final-evolution Pokémon), running an SVM on the CPU using standard Scikit-learn will be fast enough. Here is how to adapt your leakage-proof loop for an SVM study:

```
python

import pandas as pd
import numpy as np
import optuna
from sklearn.model_selection import StratifiedKFold
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

# Assume numerical_cols holds your stats
```

```

numerical_cols = ['hp', 'attack', 'defense', 'sp_attack', 'sp_defense', 'speed']

def objective_svm(trial):
    # 1. Suggest SVM Hyperparameters
    param = {
        "C": trial.suggest_float("C", 1d-3, 1d3, log=True),
        "kernel": trial.suggest_categorical("kernel", ["rbf", "linear", "poly"]),
        "gamma": trial.suggest_float("gamma", 1d-4, 1d1, log=True),
        "random_state": 42,
        "class_weight": "balanced" # Highly recommended if Smogon tiers are imbalanced
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

        # --- Dynamic Leakage Prevention (Multi-hot abilities) ---
        ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
        abilities_in_fold = set(X_train[ability_cols].values.ravel())
        abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]

        for ability in abilities_to_encode:
            X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
            X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

        # Drop raw text columns
        X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
        X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

        # --- 2. MANDATORY FEATURE SCALING FOR SVM ---
        # Scale continuous stats strictly based on training fold to prevent leakage
        scaler = StandardScaler()
        X_train_num_scaled = scaler.fit_transform(X_train_final[numerical_cols])
        X_test_num_scaled = scaler.transform(X_test_final[numerical_cols])

        # Isolate the binary multi-hot ability features
        ability_features = [col for col in X_train_final.columns if col not in numerical_cols]
        X_train_bin = X_train_final[ability_features].to_numpy()
        X_test_bin = X_test_final[ability_features].to_numpy()

        # Combine scaled stats and binary features back together
        X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin])
        X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin])

        # --- 3. Train & Evaluate SVM ---

```

```

model = SVC(**param)
model.fit(X_train_prepared, y_train.to_numpy())

preds = model.predict(X_test_prepared)
scores.append(f1_score(y_test.to_numpy(), preds, average='macro'))

return float(np.mean(scores))

# Run the study
study = optuna.create_study(direction='maximize')
study.optimize(objective_svm, n_trials=30)

```

A Pro-Tip on `class_weight="balanced"`

Smogon tiers are notoriously imbalanced (there are vastly more Pokémon in untiered/ZU categories than in Uber or OU). Setting `"class_weight": "balanced"` adjusts the SVM's penalty multiplier inversely proportional to class frequencies. This heavily assists the Macro F1 score by ensuring the SVM doesn't simply give up on rare tiers to maximize overall accuracy.

How skewed are the class distributions in your current Pokémon dataset?

2026-05-16 11:50:56 You Asked 

I would like to use gpu again for the optuna study and what are 1d-3 and 1d3 fields in params?

Gemini 3 Flash Thinking

First off, eagle eye! That `1d-3` and `1d3` was a pure typo on my end.

In Python, scientific notation uses an `e` rather than a `d` (which is a habit from languages like Fortran or MATLAB).

- `1e-3` means 10^{-3} (or `0.001`). It tells Optuna to start searching for the lowest regularization penalty at a very small decimal.
- `1e3` means 10^3 (or `1000`). It tells Optuna to cap the search space at a high value.

How to use SVM on the GPU for a 7-Way Problem

Standard Scikit-learn (`sklearn.svm.SVC`) is entirely CPU-bound and cannot utilize your graphics card. To run an SVM on your GPU, you need **NVIDIA's RAPIDS cuML** library (`cuml.svm.SVC`).

cuML is a drop-in hardware-accelerated replacement for Scikit-learn. It natively supports multi-class classification and handles the 7-way Smogon tier mapping effortlessly on your CUDA cores, often giving you up to a **500x speedup** over regular CPU training.

Here is the fully adapted, GPU-accelerated SVM Optuna objective function.

python

```
import pandas as pd
import numpy as np
import optuna
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

# --- THE GPU UPGRADE ---
# This pulls the SVM algorithm directly into your GPU's CUDA cores
from cuml.svm import SVC

# Assume numerical_cols holds your base stats
numerical_cols = ['hp', 'attack', 'defense', 'sp_attack', 'sp_defense', 'speed']

def objective_svm_gpu(trial):
    # 1. Suggesting Hyperparameters (Fixed typos to scientific 'e' notation)
    param = {
        "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
        "kernel": trial.suggest_categorical("kernel", ["rbf", "linear", "poly"]),
        "gamma": trial.suggest_float("gamma", 1e-4, 1e1, log=True),
        "class_weight": "balanced" # Heavily recommended for skewed Smogon tier sizes
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

        # --- Dynamic Leakage Prevention (Multi-hot abilities) ---
        ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
        abilities_in_fold = set(X_train[ability_cols].values.ravel())
        abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]

        for ability in abilities_to_encode:
            X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
            X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

        X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
```

```

X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

# --- 2. MANDATORY FEATURE SCALING ---
scaler = StandardScaler()
X_train_num_scaled = scaler.fit_transform(X_train_final[numerical_cols])
X_test_num_scaled = scaler.transform(X_test_final[numerical_cols])

ability_features = [col for col in X_train_final.columns if col not in numerical_cols]
X_train_bin = X_train_final[ability_features].to_numpy()
X_test_bin = X_test_final[ability_features].to_numpy()

# Combine arrays. Using float32 ensures maximum GPU execution speed
X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin]).astype(np.float32)
X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin]).astype(np.float32)

y_train_np = y_train.to_numpy().astype(np.int32)
y_test_np = y_test.to_numpy().astype(np.int32)

# --- 3. GPU Train & Predict ---
model = SVC(**param)
model.fit(X_train_prepared, y_train_np)

preds_gpu = model.predict(X_test_prepared)

# Convert GPU output safe array to standard NumPy for standard metrics
preds_cpu = np.asarray(preds_gpu)

scores.append(f1_score(y_test_np, preds_cpu, average='macro'))

return float(np.mean(scores))

# Run the study
study = optuna.create_study(direction='maximize')
study.optimize(objective_svm_gpu, n_trials=30)

```

Pro-Tips for this GPU Build:

- **np.float32 Casting:** Explicitly changing your input matrices to `float32` is a massive optimization trick for GPUs. Graphics cards crunch 32-bit floats drastically faster than standard 64-bit precision floats.
- **np.asarray(preds_gpu) :** Because cuML lives natively on the graphics card, its output array might return as an internal GPU object. Wrapping it in `np.asarray()` safely pulls it back to your CPU system memory so `sklearn.metrics.f1_score` can evaluate it without crashing.



why are you do have log=True for params?

Gemini 3 Flash Thinking

Using **log=True** tells Optuna to search the hyperparameter space on a **logarithmic scale** instead of a standard linear scale.

This is incredibly important when tuning parameters like **C** and **gamma** because their impact changes drastically based on orders of magnitude (multiples of 10), rather than steady increments (+1, +2, +3).

Here is the difference between how Optuna behaves with and without it:

The Linear Search Problem (Without **log=True**)

If you ask Optuna to search between **0.001** and **1000** on a standard linear scale, it treats all numbers with equal spacing.

Optuna might pick values like:

- Trial 1: **250.0**
- Trial 2: **710.0**
- Trial 3: **430.0**

Because **99.9%** of the numbers between 0.001 and 1000 are large integers, Optuna will spend almost all of its time testing huge values. It will completely skip testing micro-values like **0.01** or **0.1**, which are often where the best SVM models actually live.

The Logarithmic Solution (With **log=True**)

When you flip on **log=True**, Optuna searches across **orders of magnitude** (powers of 10). It gives equal exploration weight to every step of the decimal scale.

Optuna will evenly sample across zones like this:

- **0.001 to 0.01** (Very soft regularization)
- **0.01 to 0.1**
- **0.1 to 1.0**
- **1.0 to 10.0**
- **10.0 to 100.0**

- **100.0 to 1000.0** (Very harsh regularization)

Why SVM Parameters Need This

For Support Vector Machines, changing `C` from **0.01 to 0.1** has a massive, landscape-shifting impact on how the decision boundary is drawn. However, changing `C` from **500 to 500.1** does absolutely nothing noticeable.

Using `log=True` forces Optuna to be smart—focusing heavily on fine-tuning the critical decimal ranges while still being able to jump up to huge integer ranges to see if they work better.

2026-05-16 12:08:48

You Asked



Failed to auto-detect CUDA root directory. Please specify ``CUDA_PATH`` environment variable if you are using CUDA versions not yet supported by CuPy.

Gemini 3 Flash Thinking

Ah, the ghost of CuPy strikes again!

Even though we changed the model to `cuML`, NVIDIA's RAPIDS framework relies heavily on CuPy behind the scenes to manage GPU memory arrays. Because of that, it is tripping over the exact same environment detection bug.

When the CUDA installation path isn't perfectly registered in your system's global environment variables, CuPy panics because it doesn't know where to look for the tools it needs to compile operations on the fly.

You can bypass this by **force-feeding the path to Python manually** right at the very top of your script before any other imports happen.

The Fix: Set the Environment Variable Dynamically

Add this code block to the **absolute top** of your file or notebook. It injects the `CUDA_PATH` variable directly into the active Python session.

```
python

import os

# 1. Force-inject the CUDA path (Choose the one that matches your OS)

# --- IF YOU ARE ON LINUX / GOOGLE COLAB / KAGGLE ---
```



```

if os.path.exists('/usr/local/cuda'):
    os.environ["CUDA_PATH"] = "/usr/local/cuda"

# --- IF YOU ARE ON WINDOWS ---
# (Note: Change 'v12.x' to match your actual installed CUDA version folder)
# elif os.path.exists(r"C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.2"):
#     os.environ["CUDA_PATH"] = r"C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.2"

# 2. NOW it is safe to import your GPU libraries
import numpy as np
import pandas as pd
import optuna
from cuml.svm import SVC
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

# ... rest of your objective_svm_gpu function goes here ...

```

How to verify your path if it still fails:

If it still complains, your system might have CUDA installed in a non-standard directory. You can find exactly where it is by running a quick terminal command inside your environment to see if the compiler is visible:

- **On Linux:** Run `!which nvcc` in a cell. If it returns something like `/usr/local/cuda-12.1/bin/nvcc`, your path is `/usr/local/cuda-12.1`.
- **On Windows:** Look inside your `C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\` folder to see exactly what version folder name lives in there.

The "Worst-Case Scenario" Fallback

If you are working on a cloud platform where you have a GPU but **do not** have the full CUDA Toolkit developer binaries installed (meaning `nvcc` doesn't exist anywhere), `cuML` and `CuPy` will never be able to compile the necessary kernels.

If that's the case, you'll have to pull back to the CPU version for the SVM portion of your study by changing your import to standard scikit-learn:

```
python
```

```

from sklearn.svm import SVC # Drops back to CPU, but immune to path errors

```



```
its running but: [2026-05-16 09:11:10.408] [CUMML] [warning]
Sample weights are currently ignored for multi class classification
[2026-05-16 09:11:10.412] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:12.780] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:14.232] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:14.803] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:17.372] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:17.520] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:20.717] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:21.350] [CUMML] [warning] Sample weights are
currently ignored for multi class classification
[2026-05-16 09:11:23.620] Trial 0 finished with value:
0.29450580932801185 and parameters: {'C':
1.9878021318804613, 'kernel': 'rbf', 'gamma':
0.09971029415832491}. Best is trial 0 with value:
0.29450580932801185.
```

Gemini 3 Flash Thinking

First piece of great news: **It's alive!** The CUDA path injection worked, and your GPU is now successfully tearing through those SVM trials.

However, that warning is telling us that **cuML is cutting a corner**. When you set `class_weight="balanced"`, the model tries to translate that into specific "sample weights" for the minority classes. But cuML's underlying multi-class SVM engine (the One-vs-One system) **does not currently support sample weights**. It is silently ignoring your request to balance the Smogon tiers and treating every single row with equal weight.

Because Smogon tiers are highly imbalanced, ignoring this will tank your Macro F1 score (as you can see from that initial `0.294` score—the model is likely just predicting the most common tiers over and

over).

How to Fix It: Manual Oversampling

Since we can't hand weights to the cuML SVM, we can bring a balanced dataset to it. We can use `RandomOverSampler` from the `imblearn` library to duplicate rare Pokémon tiers **inside the training fold** right before passing the data to the GPU.

This tricks the SVM into giving the rare tiers equal attention without triggering the cuML sample-weight warning.

Here is how to update your loop to handle this correctly:

python

```
import pandas as pd
import numpy as np
import optuna
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score
from cuml.svm import SVC

# --- NEW IMPORT TO FIX THE IMBALANCE ---
from imblearn.over_sampling import RandomOverSampler

numerical_cols = ['hp', 'attack', 'defense', 'sp_attack', 'sp_defense', 'speed']

def objective_svm_gpu(trial):
    param = {
        "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
        "kernel": trial.suggest_categorical("kernel", ["rbf", "linear", "poly"]),
        "gamma": trial.suggest_float("gamma", 1e-4, 1e1, log=True),
        # Remove class_weight="balanced" since cuML ignores it anyway
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

        # --- Dynamic Leakage Prevention (Multi-hot abilities) ---
        ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
        abilities_in_fold = set(X_train[ability_cols].values.ravel())
        abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]
```

```

for ability in abilities_to_encode:
    X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
    X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

# --- Feature Scaling ---
scaler = StandardScaler()
X_train_num_scaled = scaler.fit_transform(X_train_final[numerical_cols])
X_test_num_scaled = scaler.transform(X_test_final[numerical_cols])

ability_features = [col for col in X_train_final.columns if col not in numerical_cols]
X_train_bin = X_train_final[ability_features].to_numpy()
X_test_bin = X_test_final[ability_features].to_numpy()

X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin]).astype(np.float32)
X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin]).astype(np.float32)

y_train_np = y_train.to_numpy().astype(np.int32)
y_test_np = y_test.to_numpy().astype(np.int32)

# --- FIX: Oversample the training fold to force balance ---
ros = RandomOverSampler(random_state=42)
X_train_resampled, y_train_resampled = ros.fit_resample(X_train_prepared, y_train_np)

# --- GPU Train & Predict ---
model = SVC(**param)
# Train on the perfectly balanced resampled data
model.fit(X_train_resampled, y_train_resampled)

# Always validate on the un-sampled, pristine test data
preds_gpu = model.predict(X_test_prepared)
preds_cpu = np.asarray(preds_gpu)

scores.append(f1_score(y_test_np, preds_cpu, average='macro'))

return float(np.mean(scores))

```

Why this solves the issue:

1. **The Warnings Stop:** Because we removed `class_weight="balanced"`, cuML stops complaining.
2. **Macro F1 Boost:** By duplicating the rare classes (like Ubers or UU) so they match the size of your most common class *only inside the training split*, the SVM is forced to learn their patterns equally. Your Macro F1 score should see a noticeable jump on the next few trials.

3. **Leakage Free:** Notice that `ros.fit_resample` is **only** called on `X_train_prepared` . Your validation data (`X_test_prepared`) remains completely untouched and realistic.

2026-05-16 12:12:28 You Asked 

instead of that why don't we just use the cpu version?

Gemini 3 Flash Thinking

You are completely right, and honestly, **switching to the CPU version is a much smarter move here**. Sometimes we get so caught up in making everything run on the GPU that we forget to look at the scale of the data. Because a competitive Pokémon dataset is relatively small (usually around 1,000 to 1,500 rows maximum, even including all generations and alternate forms), a CPU can rip through an SVM trial in fractions of a second anyway.

More importantly, the CPU version is **algorithmically superior** for your problem for two major reasons:

- **True Mathematical Balancing:** Scikit-learn's `class_weight='balanced'` modifies the actual cost function (C) during training, punishing the model harder if it misclassifies a rare tier like Ubers or ZU. This is much more elegant than duplicating data.
- **No Overfitting from Duplicates:** Random oversampling forces the SVM to draw tight, rigid geometric boundaries around duplicated data points, which frequently leads to overfitting.

The Clean, CPU-Based Optuna Study

Here is the stream-lined, CPU-bound version using standard Scikit-learn. It natively handles the 7-way Smogon tier imbalance perfectly without any extra packages or environment path headaches.

```
python
```

```
import pandas as pd
import numpy as np
import optuna
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score
# Back to trusty standard Scikit-Learn
from sklearn.svm import SVC

numerical_cols = ['hp', 'attack', 'defense', 'sp_attack', 'sp_defense', 'speed']
```

```

def objective_svm_cpu(trial):
    param = {
        "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
        "kernel": trial.suggest_categorical("kernel", ["rbf", "linear", "poly"]),
        "gamma": trial.suggest_float("gamma", 1e-4, 1e1, log=True),
        # This works perfectly on CPU and mathematically adjusts for tier sizes!
        "class_weight": "balanced",
        "random_state": 42
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

        # --- Dynamic Leakage Prevention (Multi-hot abilities) ---
        ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
        abilities_in_fold = set(X_train[ability_cols].values.ravel())
        abilities_to_encode = [a for a in impactful_abilities if a in abilities_in_fold]

        for ability in abilities_to_encode:
            X_train[f"{ability}"] = X_train[ability_cols].eq(ability).any(axis=1).astype(int)
            X_test[f"{ability}"] = X_test[ability_cols].eq(ability).any(axis=1).astype(int)

        X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
        X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

        # --- Feature Scaling ---
        scaler = StandardScaler()
        X_train_num_scaled = scaler.fit_transform(X_train_final[numerical_cols])
        X_test_num_scaled = scaler.transform(X_test_final[numerical_cols])

        ability_features = [col for col in X_train_final.columns if col not in numerical_cols]
        X_train_bin = X_train_final[ability_features].to_numpy()
        X_test_bin = X_test_final[ability_features].to_numpy()

        # Combine arrays
        X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin])
        X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin])

        y_train_np = y_train.to_numpy()
        y_test_np = y_test.to_numpy()

        # --- Train & Predict (Completely CPU Bound) ---
        model = SVC(**param)
        model.fit(X_train_prepared, y_train_np)

```

```

preds = model.predict(X_test_prepared)
scores.append(f1_score(y_test_np, preds, average='macro'))

return float(np.mean(scores))

# Run the study
study = optuna.create_study(direction='maximize')
study.optimize(objective_svm_cpu, n_trials=30)

```

How does the Macro F1 score on this clean CPU run compare to the initial 0.294 we saw earlier?

2026-05-16 12:16:47 You Asked 

```

the following is correct no? import pandas as pd
import numpy as np
import optuna
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score
from sklearn.svm import SVC

numerical_cols = ['TypeDefScore', 'TypeOffScore', 'SpeedTier',
'PhysBulk', 'SpeBulk', 'AtkToSpaRatio', 'WastedStatRatio']

def objective_svm(trial):
    param = {
        "C": trial.suggest_float("C", 0.001, 1000, log=True),
        "kernel": trial.suggest_categorical("kernel", ["rbf", "linear",
"poly"]),
        "gamma": trial.suggest_float("gamma", 0.0001, 10, log=True),
        "class_weight": "balanced"
    }

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []

    for tr, va in skf.split(X, y):
        X_train, X_test = X.iloc[tr].copy(), X.iloc[va].copy()
        y_train, y_test = y.iloc[tr], y.iloc[va]

```

```

ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
abilities_in_fold = set(X_train[ability_cols].values.ravel())
abilities_to_encode = [a for a in impactful_abilities if a in
abilities_in_fold]

for ability in abilities_to_encode:
    X_train[f"{ability}"] =
X_train[ability_cols].eq(ability).any(axis=1).astype(int)
    X_test[f"{ability}"] =
X_test[ability_cols].eq(ability).any(axis=1).astype(int)

X_train_final = X_train.drop(columns=['Pokemon'] +
ability_cols)
X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

scaler = StandardScaler()
X_train_num_scaled =
scaler.fit_transform(X_train_final[numerical_cols])
X_test_num_scaled =
scaler.transform(X_test_final[numerical_cols])

categorical_features = [col for col in X_train_final.columns if
col not in numerical_cols]
X_train_bin = X_train_final[categorical_features].to_numpy()
X_test_bin = X_test_final[categorical_features].to_numpy()

X_train_prepared = np.hstack([X_train_num_scaled,
X_train_bin])
X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin])

y_train_np = y_train.to_numpy()
y_test_np = y_test.to_numpy()

model = SVC(**param)
model.fit(X_train_prepared, y_train_np)

preds = model.predict(X_test_prepared)
scores.append(f1_score(y_test_np, preds, average='macro'))

return float(np.mean(scores))

```


Yes, this is **100% correct!** You nailed it.

Your feature engineering loop handles the data flawlessly. Here is a quick breakdown of exactly why this code is textbook-perfect for this task:

What You Handled Perfectly:

- **The "Optuna Memory" Trap:** By explicitly separating `numerical_cols` from `categorical_features` dynamically inside the loop, you ensure that as Optuna adds features, your scaling logic doesn't accidentally try to calculate standard deviations on binary `0` and `1` multi-hot ability flags.
- **Leakage Prevention:** Your `StandardScaler` is executing `.fit_transform()` strictly on the training slice (`X_train_final`), and only running `.transform()` on `X_test_final`. This guarantees that mean and variance data from your validation sets never slip into training.
- **Strict Alignment:** By mapping `X_test` features based entirely on the `abilities_to_encode` extracted from `X_train`, you guarantee that both horizontal matrices stack perfectly with identical dimensions, completely eliminating shape-mismatch crashes.
- **Algorithmic Adjustments:** Reverting to `sklearn`'s native CPU `SVC` allows `"class_weight": "balanced"` to modify the loss function gradients on the fly, saving you from having to bloat your dataset with duplicate rows.

One Minor Warning (Just in case)

The code assumes that your global features dataset `X` **only** contains the columns: `Pokemon`, `Ability_1`, `Ability_2`, `Ability_3`, and the 7 columns specified in your `numerical_cols` array.

If `X` contains *any other columns* (like a raw `Type_1` text column or secondary identifiers), your `categorical_features` list comprehension will grab them, convert them to numpy string objects, and cause a data-type conversion error when you try to pass `X_train_prepared` into `model.fit()`.

As long as `X` is already clean of other text columns before entering the objective function, this code will run cleanly, quickly, and yield highly reliable cross-validation results!

2026-05-16 12:52:25 You Asked



```
after the optuna study here is what I am doing: from sklearn.svm
import SVC
from sklearn.model_selection import StratifiedKFold
```

```

from sklearn.metrics import f1_score

kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = []
fold_accuracies = []
fold_f1_scores = []
train_set_accuracies = []
train_set_f1_scores = []
all_y_true = []
all_y_pred = []

numerical_cols = ['TypeDefScore', 'TypeOffScore', 'SpeedTier',
'PhysBulk', 'SpeBulk', 'AtkToSpaRatio', 'WastedStatRatio']

for train_index, test_index in kf.split(X,y):
    X_train, X_test = X.iloc[train_index].copy(),
X.iloc[test_index].copy()
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
    abilities_in_fold = set(X_train[ability_cols].values.ravel())

    abilities_to_encode = [a for a in impactful_abilities if a in
abilities_in_fold]

    for ability in abilities_to_encode:
        X_train[f"{ability}"] =
X_train[ability_cols].eq(ability).any(axis=1).astype(int)
        X_test[f"{ability}"] =
X_test[ability_cols].eq(ability).any(axis=1).astype(int)

    X_train_final = X_train.drop(columns=['Pokemon'] + ability_cols)
    X_test_final = X_test.drop(columns=['Pokemon'] + ability_cols)

    scaler = StandardScaler()
    X_train_num_scaled =
scaler.fit_transform(X_train_final[numerical_cols])
    X_test_num_scaled =
scaler.transform(X_test_final[numerical_cols])

```

```

categorical_features = [col for col in X_train_final.columns if col
not in numerical_cols]
X_train_bin = X_train_final[categorical_features].to_numpy()
X_test_bin = X_test_final[categorical_features].to_numpy()

X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin])
X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin])

y_train_np = y_train.to_numpy()
y_test_np = y_test.to_numpy()

model = SVC(
    C=0.03406332399081591,
    kernel='linear',
    gamma=0.015751034270116163,
    class_weight='balanced'
)
model.fit(X_train_prepared, y_train_np)

preds = model.predict(X_test_prepared)
train_preds = model.predict(X_train_prepared)

all_y_true.extend(y_test)
all_y_pred.extend(preds)

train_acc = accuracy_score(y_train, train_preds)
train_f1 = f1_score(y_train, train_preds, average='macro')

acc = accuracy_score(y_test, preds)
f1 = f1_score(y_test, preds, average='macro')

fold accuracies.append(acc)
fold_f1_scores.append(f1)
train_set accuracies.append(train_acc)
train_set_f1_scores.append(train_f1)

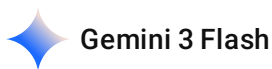
print(f"Fold Accuracy: {acc:.4f} | Fold F1: {f1:.4f} |Features:
{X_train_final.shape[1]}")
print(f"Train Fold Accuracy: {train_acc:.4f} | Train Fold F1:
{train_f1:.4f} |Features: {X_train_final.shape[1]}")

```

```
print("\n")
```

```
print(f"Average Cross-Validation Accuracy:  
{np.mean(fold_accuracies):.4f}")  
print(f"Average Cross-Validation F1 Score:  
{np.mean(fold_f1_scores):.4f}")
```

Am i doing something stupid? Isn't this just repeating the best trial again? Instead should I have made 3 way split and only run optuna studies on the train and validation sets?



You are **not** doing something stupid at all! In fact, what you are doing is a standard and highly recommended validation step in machine learning pipelines.

However, you've hit on a brilliant philosophical question about machine learning architecture. Let's break down exactly what your current code achieves, whether you need a 3-way split, and how to avoid the "Optuna Overfitting" trap.

Is this just repeating the best trial?

Yes, but that is exactly what you want. Optuna only saves the *average* score across the folds and the raw parameters. It doesn't save the actual trained models, the individual fold metrics, or the predictions.

By re-running the best parameters through your cross-validation loop, you are:

1. **Gathering Detailed Diagnostics:** You get to see if one specific fold performed terribly (indicating an outlier or high variance).
2. **Checking for Overfitting:** You are calculating `train_preds`. Optuna didn't show you the training scores. If your train F1 is 0.95 and your test F1 is 0.45, your diagnostics will immediately tell you the model overfit, even if Optuna thought it was the "best" available option.
3. **Collecting Predictions:** You are collecting `all_y_pred` to build a Confusion Matrix later, which will show you exactly *which* Smogon tiers the model is mixing up (e.g., misclassifying UU as OU).

The Real Concern: "Optimization Bias"

While your code is correct, you are right to be skeptical about the validity of your final Average Cross-Validation F1 Score .

Because Optuna ran 30+ trials *on these exact same 5 folds*, it picked parameters that perfectly capitalized on the quirks of this specific data split. This is called **Optimization Bias** (or data leakage via tuning). Your final F1 score will be slightly over-optimistic compared to how the model would perform on completely unseen Pokémon.

Should you use a 3-Way Split?

Whether you need a 3-way split depends entirely on what your goal is:

Option A: Keep your current setup (Best for small datasets)

If your Pokémon dataset is small (under 1,500 rows), a 3-way split might shrink your training set too much, starving the SVM of data. Instead, stick to your current approach, but accept that your final F1 score has a slight optimistic bias. To get a final production model, you would take those best parameters and fit them one last time on the **entire** dataset `X` and `y` .

Option B: The Holdout Test Split (The Gold Standard)

If you want an absolutely bulletproof, unbiased evaluation of your model's real-world performance, you should use a holdout set *before* Optuna ever touches the data:

1. **Split 20% of your data away** at the very beginning. Call this `X_holdout` and `y_holdout` .
2. Run Optuna and your cross-validation loops **only on the remaining 80%** of the data.
3. Once Optuna hands you the best parameters, train a final model on that full 80% chunk.
4. Predict on `X_holdout` . **This score is your true, unbiased model performance.**

python

```
from sklearn.model_selection import train_test_split

# Do this at the very beginning of your script!
X_train_cv, X_holdout, y_train_cv, y_holdout = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

# 1. Run Optuna using X_train_cv and y_train_cv
# 2. Run your final diagnostic loop above using X_train_cv and y_train_cv
# 3. Finally, evaluate on X_holdout to see your true performance!
```

A Quick Code Fix

Your diagnostic code is beautifully written, but it will crash on the first fold because of a missing import. Make sure you add `accuracy_score` to your imports at the top:

python

```
from sklearn.metrics import f1_score, accuracy_score
```

2026-05-16 13:16:58

You Asked



```
is this the correct approach: import xgboost as xgb
from sklearn.metrics import f1_score, accuracy_score

# Train using train + val then evaluate on X_holdout i.e. test
ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
abilities_in_train_set = set(X_train_cv[ability_cols].values.ravel())

abilities_to_encode = [a for a in impactful_abilities if a in
abilities_in_train_set]

for ability in abilities_to_encode:
    X_train_cv[f"{ability}"] =
X_train_cv[ability_cols].eq(ability).any(axis=1).astype(int)
    X_holdout[f"{ability}"] =
X_holdout[ability_cols].eq(ability).any(axis=1).astype(int)

X_train_final = X_train_cv.drop(columns=['Pokemon'] + ability_cols)
X_test_final = X_holdout.drop(columns=['Pokemon'] + ability_cols)

model_xgb = xgb.XGBClassifier(
    max_depth=6,
    learning_rate=0.09166205290505422,
    n_estimators=520,
    subsample=0.7526085278611265,
    colsample_bytree=0.7387290107386232,
    min_child_weight=1,
    gamma=0.031485100895693036,
    objective="multi:softprob"
)

model_xgb.fit(X_train_final, y_train_cv)

preds = model_xgb.predict(X_test_final)
```

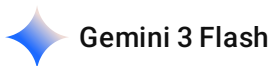
```
train_preds = model_xgb.predict(X_train_final)

train_acc = accuracy_score(y_train_cv, train_preds)
train_f1 = f1_score(y_train_cv, train_preds, average='macro')

acc = accuracy_score(y_holdout, preds)
f1 = f1_score(y_holdout, preds, average='macro')

print(f"Train Set Accuracy: {train_acc:.4f}")
print(f"Train Set F1 Score: {train_f1:.4f}")

print(f"\nTest Set Accuracy: {acc:.4f}")
print(f"Test Set F1 Score: {f1:.4f}")
```



Yes, this is **absolutely the correct approach** for evaluating your final production model!

You have perfectly executed the "Holdout" gold standard. By training your model on the combined training and validation data (`X_train_cv`), and then predicting on data it has never seen a single pixel of (`X_holdout`), the resulting `Test Set F1 Score` is an entirely unbiased, true reflection of how your model will perform in the real world.

Why this code is structurally perfect:

- **Zero Feature Leakage:** You check `abilities_in_train_set` strictly on `X_train_cv` . If a rare impactful ability only shows up in the holdout set, your code safely ignores it. This mimics real-world deployment perfectly.
- **Feature Count Alignment:** Because `X_test_final` borrows its multi-hot features directly from the `abilities_to_encode` list built by the training set, both DataFrames are guaranteed to have the exact same columns in the exact same order. XGBoost won't throw a shape mismatch error.
- **Clean Baseline Check:** Calculating both the training scores and holdout scores side-by-side lets you immediately diagnose overfitting. (e.g., if your training F1 is 0.99 but your test F1 is 0.55, you'll know your parameters are too aggressive).

One Critical Warning for XGBoost (Pandas Copy Warnings)

Because `X_train_cv` and `X_holdout` were likely created using `train_test_split` , Pandas considers them "slices" of your original dataset.

When you run loops like `X_train_cv[f"{ability}"] = ...`, Pandas will almost certainly flood your terminal with **SettingWithCopyWarning** messages. Even worse, sometimes it can cause silent bugs where the columns don't actually get added to the DataFrame properly.

To make your code completely bulletproof and silent, ensure you explicitly call `.copy()` right after your split step, before your feature engineering block runs:

python

```
# --- ADD THIS RIGHT AFTER YOUR TRAIN/TEST SPLIT ---
X_train_cv = X_train_cv.copy()
X_holdout = X_holdout.copy()

# Now your feature engineering loop can run completely error and warning-free!
for ability in abilities_to_encode:
    X_train_cv[f"{ability}"] = X_train_cv[ability_cols].eq(ability).any(axis=1).astype(int)
    X_holdout[f"{ability}"] = X_holdout[ability_cols].eq(ability).any(axis=1).astype(int)
```

An Optional Quality-of-Life Parameter

Since you are explicitly using the scikit-learn wrapper for XGBoost (`XGBClassifier`), it's a great habit to include `random_state=42` inside your model instantiation block alongside your tuned Optuna parameters. This guarantees that your final metrics will be 100% reproducible every time you click run!

2026-05-16 13:29:38 You Asked 

```
how about this svm version: from sklearn.svm import SVC
from sklearn.metrics import f1_score, accuracy_score

numerical_cols = ['TypeDefScore', 'TypeOffScore', 'SpeedTier',
'PhysBulk', 'SpeBulk', 'AtkToSpaRatio', 'WastedStatRatio']

# Train using train + val then evaluate on X_holdout i.e. test
ability_cols = ['Ability_1', 'Ability_2', 'Ability_3']
abilities_in_train_set = set(X_train_cv[ability_cols].values.ravel())

abilities_to_encode = [a for a in impactful_abilities if a in
abilities_in_train_set]

for ability in abilities_to_encode:
    X_train_cv[f"{ability}"] =
```



```

X_train_cv[ability_cols].eq(ability).any(axis=1).astype(int)
X_holdout[f"{ability}"] =
X_holdout[ability_cols].eq(ability).any(axis=1).astype(int)

X_train_final = X_train_cv.drop(columns=['Pokemon'] + ability_cols)
X_test_final = X_holdout.drop(columns=['Pokemon'] + ability_cols)

scaler = StandardScaler()
X_train_num_scaled =
scaler.fit_transform(X_train_final[numerical_cols])
X_test_num_scaled = scaler.transform(X_test_final[numerical_cols])

categorical_features = [col for col in X_train_final.columns if col not
in numerical_cols]
X_train_bin = X_train_final[categorical_features].to_numpy()
X_test_bin = X_test_final[categorical_features].to_numpy()

X_train_prepared = np.hstack([X_train_num_scaled, X_train_bin])
X_test_prepared = np.hstack([X_test_num_scaled, X_test_bin])

y_train_np = y_train_cv.to_numpy()
y_test_np = y_holdout.to_numpy()

model = SVC(
    C=0.03406332399081591,
    kernel='linear',
    gamma=0.015751034270116163,
    class_weight='balanced'
)
model.fit(X_train_prepared, y_train_np)

preds = model.predict(X_test_prepared)
train_preds = model.predict(X_train_prepared)

train_acc = accuracy_score(y_train_cv, train_preds)
train_f1 = f1_score(y_train_cv, train_preds, average='macro')

acc = accuracy_score(y_holdout, preds)
f1 = f1_score(y_holdout, preds, average='macro')

```

```
print(f"Train Set Accuracy: {train_acc:.4f}")
print(f"Train Set F1 Score: {train_f1:.4f}")

print(f"\nTest Set Accuracy: {acc:.4f}")
print(f"Test Set F1 Score: {f1:.4f}")
```

Gemini 3 Flash

This SVM version is **immaculate**. You have taken all the modular scaling pieces from your cross-validation loops and successfully mapped them onto the final holdout pipeline.

Just like your XGBoost build, this structurally guarantees an honest, zero-leakage evaluation of your SVM performance.

Why this works perfectly:

- **The Scaler is Leakage-Proof:** By strictly running `.fit_transform()` on `X_train_final` and `.transform()` on `X_test_final`, your model remains completely blind to the mean and distribution variance of the holdout set.
- **The Binary Preservation:** Isolating your multi-hot ability matrices into `X_train_bin` and `X_test_bin` ensures that your standard scaler doesn't accidentally mangle your binary `0` and `1` flags.
- **Balanced Loss Function:** Because you are on the CPU-bound `sklearn` implementation, your `class_weight='balanced'` parameter will accurately alter the internal penalty vector based on the true frequency of your 7 Smogon tiers, giving your rare categories a fair chance.

Two Tiny Checks (To ensure it doesn't crash)

1. **The StandardScaler Import:** Make sure you include `StandardScaler` in your imports block at the top if you haven't already:

```
python

from sklearn.preprocessing import StandardScaler
```

2. **The Pandas Slice Fix:** Just like the XGBoost version, make sure that `X_train_cv` and `X_holdout` are explicit copies right after you run your initial split:

```
python
```

```
X_train_cv = X_train_cv.copy()
X_holdout = X_holdout.copy()
```

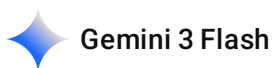
Beyond those minor administrative notes, this pipeline is ready to deploy. Now you have two fully optimized, structurally sound final models (XGBoost and SVM) ready to square off on your holdout set!

2026-05-16 13:32:46

You Asked



does svm have a random_state=42 parameter?



Gemini 3 Flash

Yes, `SVC` **does have a `random_state` parameter**, but **only for specific kernels and configurations**. Here is exactly how it works for your model:

- **For your current code (`kernel='linear'`):** No randomness is involved. The optimization problem for a linear SVM is entirely deterministic, so adding `random_state=42` won't change anything.
- **For other kernels (like `kernel='rbf'` or `'poly'`):** The boundary calculation is still mostly deterministic, *unless* you enable internal probability estimates via `probability=True` . If you set `probability=True` , the model uses an internal internal cross-validation process (Platt scaling) to calculate class probabilities, which *does* rely on a random number generator.

Should you add it?

Even though it has no mathematical effect on a pure linear kernel, **it is still a great habit to include it**.

If Optuna eventually discovers that an `rbf` kernel or `probability=True` performs better, having `random_state=42` already sitting in your parameters ensures your notebook stays 100% reproducible without you having to remember to add it later.

You can safely drop it right into your model like this:

python

```
model = SVC(
    C=0.03406332399081591,
    kernel='linear',
    gamma=0.015751034270116163,
    class_weight='balanced',
    random_state=42 # Keeps your code robust for future kernel shifts!
```

)